#include <iostream>

using namespace std;

struct Node { int data; Node* left; Node* right; };

Node* createNode(int value) { Node* newNode = new Node; newNode->data = value; newNode->left = nullptr; newNode->right = nullptr; return newNode; }

Node* insert(Node* root, int value) { if (root == nullptr) return createNode(value);

```
if (value < root->data)
    root->left = insert(root->left, value);
else
    root->right = insert(root->right, value);

return root;


}
```

void inorder(Node* root) { if (root == nullptr) return;

```
inorder(root->left);
cout << root->data << " ";
inorder(root->right);


}
```

void mirror(Node* root) { if (root == nullptr) return;

```
Node* temp = root->left;
root->left = root->right;
root->right = temp;

mirror(root->left);
mirror(root->right);


}
```

void deleteTree(Node* root) { if (root == nullptr) return;

```
deleteTree(root->left);
deleteTree(root->right);
delete root;


}
```

int main() { Node* root = nullptr; int n, value;

```
cout << "Enter number of nodes: ";
cin >> n;

if (n <= 0) {
    cout << "Number of nodes must be positive.";
    return 0;
}

cout << "Enter node values: ";
for (int i = 0; i < n; i++) {
    cin >> value;
    root = insert(root, value);
}

cout << "\nOriginal tree (Inorder): ";
inorder(root);

mirror(root);
cout << "\nMirror image (Inorder): ";
inorder(root);

mirror(root);
cout << "\nTree after mirroring twice: ";
inorder(root);

cout << "\n\nThe original tree is restored after two mirror
operations.\n";

deleteTree(root);
```

```
return 0;

}
```